

Technical Report: Exploring SocketCAN – Software-Only CAN/CAN FD Communication on Linux

1. Introduction

Controller Area Network (CAN) is widely used in automotive systems for communication between Electronic Control Units (ECUs). This project investigates SocketCAN, the Linux framework for CAN communication, and demonstrates CAN and CAN FD communication without physical CAN hardware.

The complete system was developed in Ubuntu Linux running in VirtualBox using a Virtual CAN interface (`vcn0`). Three software ECUs were implemented using Python: Vehicle ECU, Dashboard ECU, and Logger ECU. The project also demonstrates message filtering, transmission-rate variation, ECU failure detection, basic diagnostics, and CAN FD communication.

2. SocketCAN Overview

SocketCAN is the CAN networking framework provided by the Linux kernel. It allows CAN interfaces to be treated as network interfaces and provides applications with a standard socket-based method of communicating with CAN networks.

Unlike vendor-specific CAN APIs, SocketCAN provides a common Linux interface that is independent of a particular CAN hardware manufacturer. This makes applications easier to develop, test, monitor and integrate.

For this project, a Virtual CAN (`vcn0`) interface was created. It behaves like a CAN network interface but does not require a physical CAN controller, transceiver or CAN bus.

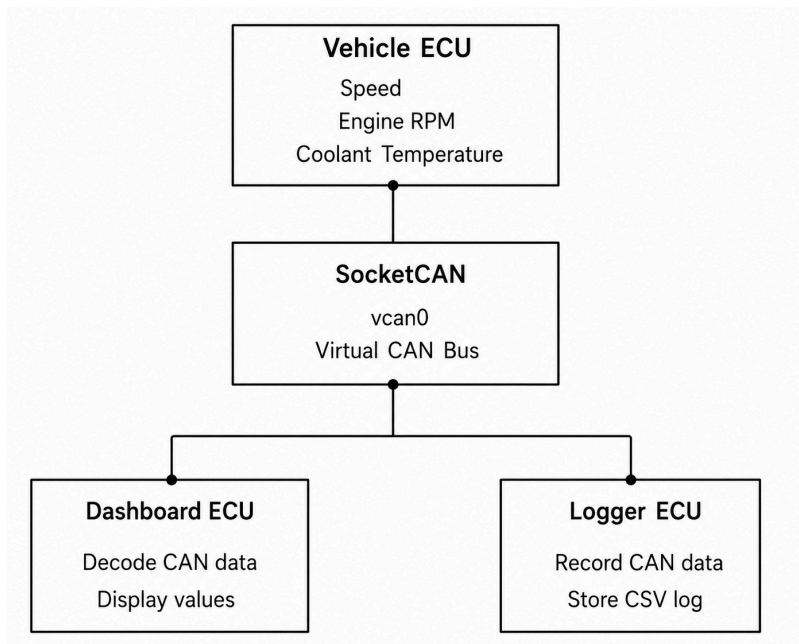
The main advantages observed during the project were:

- Software-only CAN communication without physical hardware.
- Standard Linux networking interface.
- Multiple applications can observe the same CAN traffic.
- Support for Classical CAN and CAN FD.
- Easy integration with Python using the `python-can` library.
- Availability of Linux CAN utilities such as `candump` and `cansend`.
- Suitable for development and testing before physical hardware is available.

The software environment used for the project consisted of Ubuntu Linux, VirtualBox, SocketCAN, Virtual CAN, Python, `python-can` and `can-utils`.

3. System Architecture

The system consists of three software nodes connected through the virtual CAN interface.



The Vehicle ECU generates vehicle parameters and periodically transmits them as CAN frames.

The Dashboard ECU receives CAN frames, identifies them using their CAN IDs, decodes the payload and displays the latest vehicle information. It also supports filtering for all messages, Speed messages only and RPM messages only.

The Logger ECU independently observes CAN traffic and records the timestamp, CAN ID, DLC and payload data in a CSV file.

4. CAN Message Definitions

Three standard CAN identifiers were defined for the vehicle data.

CAN ID	Message	DLC	Data Format	Unit
0x100	Vehicle Speed	2	Unsigned integer	km/h
0x101	Engine RPM	2	Unsigned integer	rpm

0x102	Coolant Temperature	1	Unsigned integer	°C
-------	---------------------	---	------------------	----

Vehicle Speed with CAN ID 0x100 is encoded as a two-byte little-endian unsigned integer.

Engine RPM with CAN ID 0x101 is also encoded as a two-byte little-endian unsigned integer.

Coolant Temperature with CAN ID 0x102 is encoded as a single unsigned byte.

The selected operating ranges were:

- Vehicle Speed: 0–120 km/h
- Engine RPM: 800–5000 rpm
- Coolant Temperature: 20–120 °C

The Dashboard ECU reverses the encoding process to recover the original values from the received CAN payload.

5. Results of Learning Challenges

Challenge 1 – Traffic Observation

The Vehicle ECU, Dashboard ECU and Logger ECU were operated simultaneously on vcan0.

The Vehicle ECU transmitted CAN frames while the Dashboard displayed the decoded information and the Logger recorded the same traffic.

The experiment demonstrated that multiple applications can observe the same SocketCAN traffic simultaneously. Activating the Logger did not interrupt Dashboard operation.

This demonstrates the usefulness of SocketCAN for applications such as monitoring, logging and diagnostics.

Challenge 2 – Message Filtering

The Dashboard was implemented with three filtering modes:

1. All messages
2. Speed only
3. RPM only

When All was selected, the Dashboard displayed Speed, RPM and Temperature.

When Speed only was selected, only CAN ID 0x100 was processed for display.

When RPM only was selected, only CAN ID 0x101 was processed for display.

The experiment demonstrated that CAN messages can be selectively processed based on their identifiers. The Logger continued recording CAN traffic independently of the Dashboard filtering.

Challenge 3 – Unknown Message Detection

An additional CAN identifier outside the predefined vehicle message set was considered as an unknown message.

The Logger architecture is designed to record CAN traffic without requiring knowledge of the message definition, while the Dashboard only decodes identifiers for which signal definitions are available.

This demonstrates that a CAN frame can be present on the network even when a particular application does not understand its meaning.

Challenge 4 – Transmission Rate Study

The Vehicle ECU transmission rate was varied by changing the delay between transmission cycles. Each cycle transmitted three CAN frames.

Delay	Approx. Rate	Approx. Frames/Second
1.0 s	1 Hz	3
0.5 s	2 Hz	6
0.2 s	5 Hz	15
0.1 s	10 Hz	30

The results showed that increasing the transmission rate increases the number of CAN frames generated and the amount of logging activity.

Higher transmission rates provide more frequent updates but also increase communication and processing requirements.

Challenge 5 – Node Failure Study

The Vehicle ECU was terminated while the Dashboard and Logger remained active.

The Dashboard monitored the time since the last Speed message. When no Speed message was received for more than two seconds, the Dashboard displayed:

WARNING: Vehicle ECU Offline

The Logger remained operational but did not receive any new messages from the stopped Vehicle ECU.

The Vehicle ECU was then restarted. Once Speed messages resumed, the Dashboard automatically returned to normal operation.

This demonstrated that communication loss can be detected at the application level by monitoring expected message timeouts.

Challenge 6 – CAN FD Exploration

The project was extended to support CAN FD.

Classical CAN supports a maximum payload of 8 bytes, whereas CAN FD supports payloads of up to 64 bytes.

A 16-byte CAN FD frame was successfully transmitted using CAN ID `0x300`.

Example payload:

```
11 22 33 44 55 66 77 88
99 AA BB CC DD EE FF 00
```

The CAN FD message was configured using CAN FD support in the Python CAN interface.

Feature	Classical CAN	CAN FD
Maximum payload	8 bytes	64 bytes
Larger data transfer	Limited	Supported
Flexible data phase	No	Yes
Demonstration	Standard CAN messages	16-byte FD frame

The experiment showed that CAN FD allows more information to be transmitted in a single frame. Application-level CAN FD support must also be explicitly enabled.

6. Basic Diagnostics

A basic diagnostic mechanism was implemented in the Dashboard ECU.

The Dashboard continuously monitors the Speed message with CAN ID `0x100`. A timeout threshold of two seconds was used.

If no Speed message is received within the specified duration, the Dashboard displays:

WARNING: Vehicle ECU Offline

When communication resumes, the warning is automatically cleared.

The diagnostic test successfully detected both Vehicle ECU communication loss and recovery.

7. Key Learnings

The major learnings from the project are:

1. SocketCAN provides a standard Linux interface for CAN communication and reduces dependence on vendor-specific APIs.
2. Virtual CAN enables hardware-independent development. The complete basic CAN system was developed using `vcn0` without a physical CAN controller or CAN transceiver.
3. CAN communication is message-oriented. Applications identify and interpret messages using CAN identifiers and predefined signal definitions.
4. Message filtering allows applications to process only the CAN messages relevant to their function.
5. Transmission rate affects CAN traffic. Higher message rates provide faster updates but increase communication and logging activity.
6. Message timeout detection can be used to identify communication failures between ECUs.
7. CAN FD extends Classical CAN by increasing the payload capacity from 8 bytes to 64 bytes and supporting a faster data phase.

8. Conclusion

This project successfully demonstrated software-only CAN and CAN FD communication using Linux SocketCAN. A virtual CAN interface, `vcn0`, was used to simulate a CAN network inside Ubuntu without requiring physical CAN hardware.